



Clasificación: Árboles de Decisión, Clasificadores Basados en Reglas y Sobreajuste

Introducción a Data Mining · Maestría en Explotación de Datos y Gestión
del Conocimiento · Universidad Austral

23 de junio de 2026

Integrantes del grupo

Ulises Catalano

Juan Pablo Fara

Marcos Maceo

Andres Rodriguez

Índice

Parte A. Árboles de Decisión: Impureza e Inducción	4
Ejercicio 1. Medidas de impureza y selección del atributo raíz	4
Ejercicio 2. Inducción del árbol de decisión completo	5
Ejercicio 3. Partición de un atributo continuo	7
Parte B. Evaluación de Clasificadores	9
Ejercicio 4. Matriz de confusión y métricas de desempeño	9
Parte C. Sobreajuste	12
Ejercicio 5. Diagnóstico del sobreajuste	12
Ejercicio 6. Poda mediante el error pesimista	13
Ejercicio 7. Estrategias de control del sobreajuste	15
Parte D. Clasificadores Basados en Reglas	16
Ejercicio 8. Derivación de reglas a partir de un árbol de decisión	16
Ejercicio 9. Conjunto ordenado de reglas frente a votación	18
Ejercicio 10. Generación directa de reglas: cobertura secuencial	19
Parte E. Preguntas Teóricas de Desarrollo	21
Parte F. Práctica en R	25
Ejercicio 11. Árboles de decisión con rpart	26
Ejercicio 12. Sobreajuste y poda en R	28
Ejercicio 13. Clasificador basado en reglas en R	32
Referencias	45

Parte A. Árboles de Decisión: Impureza e Inducción

El Conjunto de Datos 1 registra 14 solicitudes de crédito. Cada solicitud se describe mediante tres atributos predictores (Ingreso, Historial crediticio y Deuda) y el atributo de clase Otorga, que indica si se otorgó el crédito.

Ejercicio 1. Medidas de impureza y selección del atributo raíz

Con base en el Conjunto de Datos 1:

a) Calcule la entropía y el índice de Gini del nodo raíz, es decir, antes de realizar cualquier partición.

Las medidas de impureza del conjunto original arrojan los siguientes resultados:

Tabla 1

Medidas de impureza del nodo raíz

Medida	Valor
Entropía inicial	0,9852
Gini inicial	0,4898

Nota. Elaboración propia a partir de los datos del enunciado.

b) Para cada uno de los tres atributos candidatos (Ingreso, Historial y Deuda), calcule la entropía ponderada de los nodos hijos y la ganancia de información correspondiente.

c) Calcule asimismo el índice de Gini ponderado de cada atributo candidato y la reducción de la impureza de Gini que este produce.

Para cada atributo se calculó la ganancia de información y la reducción de impureza medida mediante el índice de Gini:

Tabla 2

Ganancia de información e índice de Gini por atributo candidato

Variable	Entropía ponderada	Ganancia de información	Gini ponderado	Reducción de impureza
Historial	0,7274	0,2578	0,3265	0,1633
Deuda	0,9740	0,0113	0,4821	0,0077
Ingreso	0,6046	0,3806	0,2857	0,2041

Nota. Elaboración propia a partir de los datos del enunciado.

d) Indique qué atributo seleccionaría como raíz del árbol según la ganancia de información. ¿Coincide la elección con la que indicaría el índice de Gini? Justifique su respuesta.

Ambos criterios buscan seleccionar el atributo que mejor separa las clases (Sí y No). La ganancia de información maximiza la reducción de entropía, mientras que el índice de Gini maximiza la reducción de impureza. En este conjunto de datos, el atributo **Ingreso** presenta los mejores valores en ambos criterios, por lo que su selección como nodo raíz del árbol de decisión coincide entre ambos métodos.

Ejercicio 2. Inducción del árbol de decisión completo

A partir del atributo raíz seleccionado en el Ejercicio 1, continúe la inducción del árbol de decisión de manera recursiva, con la ganancia de información como criterio de partición, hasta que todos los nodos hoja sean puros o no queden atributos disponibles.

a) Dibuje el árbol de decisión resultante. En cada nodo interno indique el atributo de partición y, en cada hoja, la clase predicha junto con la cantidad de instancias que contiene.

A continuación se presenta el árbol de decisión inducido a partir del Conjunto de Datos 1:

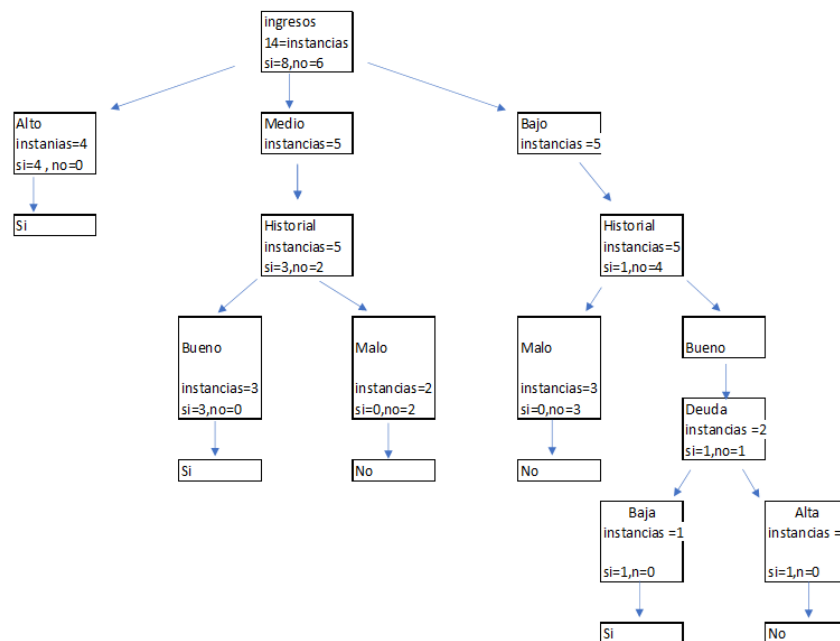


Figura 1: Árbol de decisión inducido sobre el Conjunto de Datos 1.

b) Expresar el árbol en forma de pseudocódigo, mediante una estructura de reglas anidadas "si... entonces...".

El árbol se expresa mediante la siguiente estructura de reglas anidadas:

```

SI Ingreso = "Alto" ENTONCES
    Otorga = "Sí"
SI Ingreso = "Medio" ENTONCES
    SI Historial = "Bueno" ENTONCES Otorga = "Sí"

```

```
    SI Historial = "Malo"  ENTONCES  Otorga = "No"
SI Ingreso = "Bajo" ENTONCES
    SI Historial = "Malo"  ENTONCES  Otorga = "No"
    SI Historial = "Bueno" ENTONCES
        SI Deuda = "Baja"  ENTONCES  Otorga = "Sí"
        SI Deuda = "Alta"  ENTONCES  Otorga = "No"
```

c) ¿Cuántos nodos hoja tiene el árbol? ¿Cuál es su error de resustitución, es decir, su error sobre el propio conjunto de entrenamiento?

El árbol cuenta con **6 nodos hoja**. Dado que todas las hojas son puras (cada una contiene instancias de una única clase), el error de resustitución es **0**.

Ejercicio 3. Partición de un atributo continuo

El Conjunto de Datos 2 contiene 10 registros caracterizados por un atributo continuo, el Ingreso imponible (expresado en miles de pesos), y por el atributo de clase Evade, que indica si el contribuyente evade impuestos.

a) Enumere los puntos de corte candidatos, definidos como los puntos medios de los valores consecutivos del atributo ordenados.

Los puntos de corte candidatos se determinaron a partir del punto medio entre cada par de valores consecutivos del atributo Ingreso imponible, ordenado de forma ascendente:

Tabla 3

Puntos de corte candidatos para el atributo Ingreso imponible

Intervalos	Punto medio
60 a 70	65
70 a 80	75
80 a 95	87,5
95 a 100	97,5
100 a 110	105
110 a 115	112,5
115 a 120	117,5
120 a 130	125
130 a 140	135

Nota. Elaboración propia a partir de los datos del enunciado.

b) Para cada punto de corte candidato, calcule el índice de Gini ponderado de la partición binaria resultante.

Para cada punto de corte candidato se calculó el índice de Gini de los subconjuntos generados y el índice de Gini ponderado de la partición binaria resultante:

Tabla 4

Índice de Gini ponderado por punto de corte candidato

Punto medio	Sí (<=)	No (<=)	Gini (<=)	Sí (>)	No (>)	Gini (>)	Gini ponderado
65	0	1	0	5	4	0,4938	0,4444
75	0	2	0	5	3	0,4688	0,3750
87,5	0	3	0	5	2	0,4082	0,2857
97,5	1	3	0,3750	4	2	0,4444	0,4167
105	1	4	0,3200	4	1	0,3200	0,3200
112,5	2	4	0,4444	3	1	0,3750	0,4167
117,5	3	4	0,4898	2	1	0,4444	0,4762
125	4	4	0,5000	1	1	0,5000	0,5000
135	5	4	0,4938	0	1	0	0,4444

Nota. Elaboración propia a partir de los datos del enunciado.

c) Indique el punto de corte óptimo y exprese la condición de prueba asociada al nodo.

El punto de corte óptimo es **87,5**, ya que presenta el menor índice de Gini ponderado (0,2857) entre todos los candidatos evaluados. La condición de prueba asociada al nodo es Ingreso imponible $\leq 87,5$. Esta condición divide el conjunto de datos en dos subconjuntos: los contribuyentes con ingreso imponible menor o igual a 87,5 se asignan a la rama izquierda del árbol (nodo puro de clase “No”), mientras que aquellos con ingreso superior a 87,5 se asignan a la rama derecha.

d) Explique por qué los árboles de decisión solo generan fronteras de decisión paralelas a los ejes de coordenadas.

Los árboles de decisión solo generan fronteras paralelas a los ejes porque cada nodo divide los datos con base en un único atributo a la vez. Estas particiones se realizan mediante umbrales sobre una variable, lo que produce cortes alineados con los ejes del espacio de características. Por ello, las fronteras diagonales o curvas solo pueden aproximarse mediante varias divisiones consecutivas.

Parte B. Evaluación de Clasificadores

Ejercicio 4. Matriz de confusión y métricas de desempeño

Un clasificador binario fue evaluado en un conjunto de prueba de 100 pacientes para detectar una enfermedad. Se define como clase positiva “Enfermo” y como clase negativa “Sano”. El modelo produjo 35 verdaderos positivos, 15 falsos negativos, 10 falsos positivos y 40 verdaderos negativos.

a) Construya la matriz de confusión correspondiente, identificando claramente la clase real y la predicha.

A continuación se muestra la matriz de confusión del clasificador, donde las filas representan la clase real y las columnas la clase predicha por el modelo:

Tabla 5

Matriz de confusión del clasificador

Clase real / Clase predicha	Enfermo(+) predicho	Sano(−) predicho	Total
Enfermo(+)	35 (VP)	15 (FN)	50
Sano(−)	10 (FP)	40 (VN)	50
Total	45	55	100

Nota. Elaboración propia a partir de los datos del enunciado.

El modelo identificó correctamente a 35 pacientes enfermos (verdaderos positivos) y a 40 pacientes sanos (verdaderos negativos). Sin embargo, clasificó incorrectamente a 15 pacientes enfermos como sanos (falsos negativos) y a 10 pacientes sanos como enfermos (falsos positivos).

b) Calcule la exactitud, la tasa de error, la precisión, la sensibilidad (recall), la especificidad y la medida F1.

A partir de la matriz de confusión: VP = 35, VN = 40, FP = 10, FN = 15, N = 100.

Exactitud:

$$\text{Exactitud} = \frac{VP + VN}{N} = \frac{35 + 40}{100} = 0,75$$

El modelo clasifica correctamente al 75 % de los pacientes.

Tasa de error:

$$\text{Tasa de error} = 1 - \text{Exactitud} = 0,25$$

El 25 % de las clasificaciones realizadas por el modelo son incorrectas.

Precisión:

$$\text{Precisión} = \frac{VP}{VP + FP} = \frac{35}{45} \approx 0,7778$$

Cuando el modelo predice que un paciente está enfermo, acierta aproximadamente en el 78 % de los casos.

Sensibilidad (Recall):

$$\text{Sensibilidad} = \frac{VP}{VP + FN} = \frac{35}{50} = 0,70$$

El modelo detecta correctamente el 70 % de los pacientes que realmente padecen la enfermedad.

Especificidad:

$$\text{Especificidad} = \frac{VN}{VN + FP} = \frac{40}{50} = 0,80$$

El modelo clasifica correctamente como sanos al 80 % de los pacientes que realmente no tienen la enfermedad.

Medida F1:

$$F_1 = \frac{2 \times \text{Precisión} \times \text{Sensibilidad}}{\text{Precisión} + \text{Sensibilidad}} = \frac{2 \times 0,7778 \times 0,70}{0,7778 + 0,70} \approx 0,7368$$

La medida F1 combina precisión y sensibilidad en un único valor, y penaliza los casos en que una de estas métricas es alta y la otra es baja. Un valor de 73,68 % muestra un desempeño aceptable y relativamente equilibrado entre la detección de enfermos y la confiabilidad de las predicciones positivas.

c) *Calcule el estadístico kappa de Cohen e interprete su valor.*

El coeficiente Kappa de Cohen cuantifica el grado de concordancia entre dos clasificaciones, con ajuste por el efecto del azar:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

donde P_o es el acuerdo observado y P_e es el acuerdo esperado por azar.

Acuerdo observado: $P_o = 0,75$

Acuerdo esperado por azar, a partir de las proporciones marginales:

$$P_e = \left(\frac{50}{100} \times \frac{45}{100} \right) + \left(\frac{50}{100} \times \frac{55}{100} \right) = 0,225 + 0,275 = 0,50$$

Estadístico Kappa:

$$\kappa = \frac{0,75 - 0,50}{1 - 0,50} = \frac{0,25}{0,50} = 0,50$$

Un valor de $\kappa = 0,50$ indica un nivel de concordancia moderado, superior al que se obtendría por simple azar, aunque todavía existe margen para mejorar la capacidad de clasificación del modelo.

d) Suponga que un falso negativo (no detectar a un paciente efectivamente enfermo) acarrea un costo mucho mayor que un falso positivo. ¿Qué métrica priorizaría para seleccionar el modelo? Justifique.

La métrica que debe priorizarse es la **sensibilidad** (*recall*), dado que mide la capacidad del modelo para identificar correctamente a los pacientes que realmente están enfermos. En un contexto médico, un falso negativo (no detectar a un paciente efectivamente enfermo) acarrea consecuencias clínicas graves. La sensibilidad es, por lo tanto, la métrica que mejor refleja la capacidad del modelo para reducir este tipo de error, el más crítico en este escenario.

Parte C. Sobreajuste

Ejercicio 5. Diagnóstico del sobreajuste

La Tabla 3 presenta el error de un árbol de decisión sobre el conjunto de entrenamiento y sobre un conjunto de validación independiente, en función de la complejidad del modelo, medida como el número de nodos hoja.

a) Represente gráficamente ambas curvas de error en función de la complejidad del modelo, en un mismo sistema de ejes.

A continuación se representan las curvas de error de entrenamiento y de validación en función de la complejidad del modelo, medida a través del número de nodos hoja del árbol de decisión:

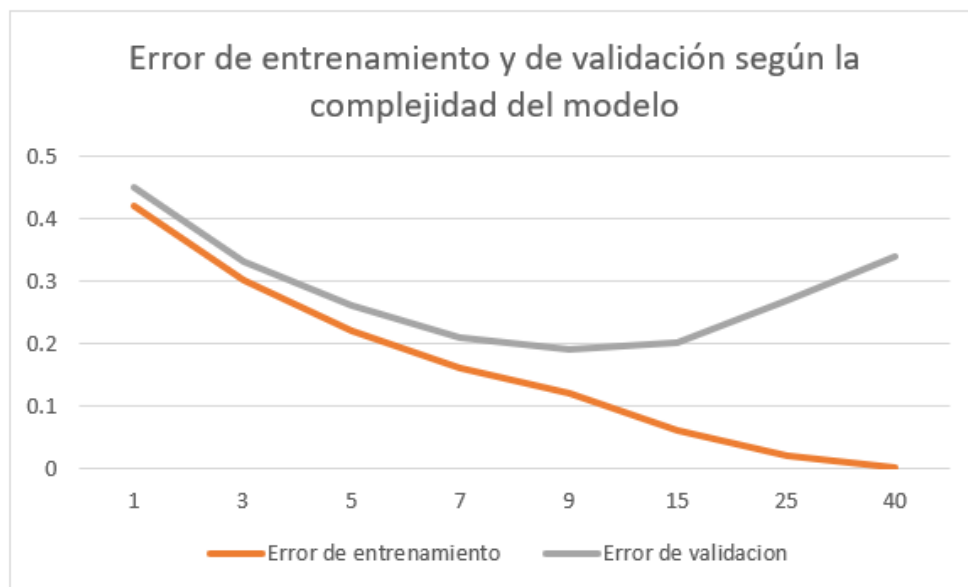


Figura 2: Curvas de error de entrenamiento y de validación según la complejidad del modelo.

b) Identifique las regiones de subajuste y de sobreajuste, y señale la complejidad óptima.

A partir de las curvas obtenidas, se observa una región de **subajuste** en los modelos de menor complejidad (entre 1 y 5 nodos hoja), donde ambos errores son relativamente altos. El **sobreajuste** comienza a evidenciarse a partir de los 15 nodos hoja, ya que el error de entrenamiento sigue disminuyendo mientras que el de validación aumenta. La **complejidad óptima** corresponde a 9 nodos hoja, dado que presenta el menor error de validación (0,19).

La siguiente figura señala las regiones de subajuste y de sobreajuste, así como el punto de complejidad óptima, correspondiente al mínimo error de validación:

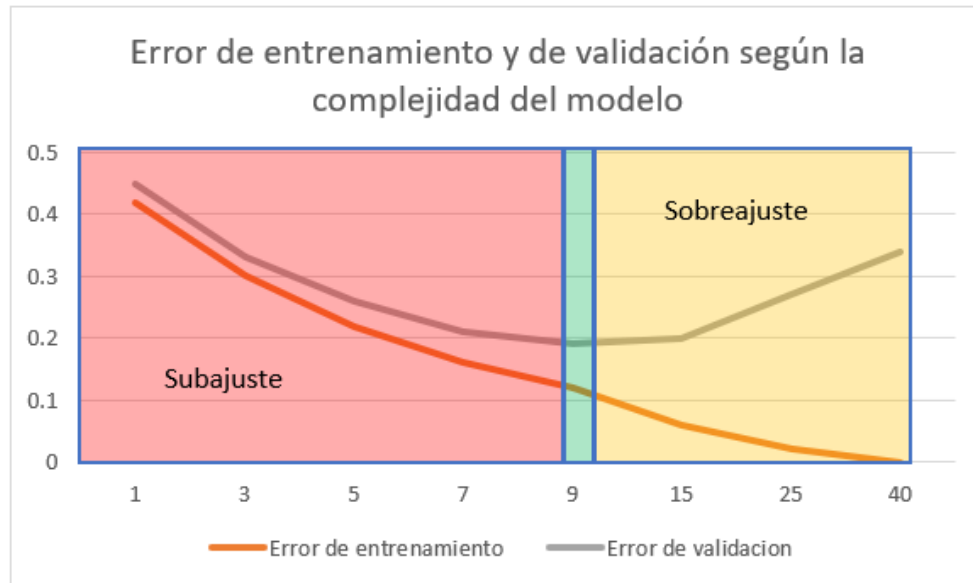


Figura 3: Regiones de subajuste y sobreajuste y punto de complejidad óptima.

c) Explique por qué el error de entrenamiento decrece de forma monótona, mientras que el error de validación describe una curva en U.

El error de entrenamiento decrece de forma monótona porque, al aumentar la complejidad del modelo, este dispone de una mayor capacidad para ajustarse a los datos de entrenamiento. En cambio, el error de validación presenta una curva en U: inicialmente disminuye al capturar mejor los patrones de los datos, pero luego aumenta debido al sobreajuste, lo que reduce la capacidad de generalización del modelo.

d) ¿Qué complejidad de modelo seleccionaría? Justifique su elección en términos del error de generalización.

Se seleccionaría el modelo con **9 nodos hoja**, ya que presenta el menor error de validación (0,19). Este modelo logra el mejor equilibrio entre el ajuste a los datos de entrenamiento y el desempeño sobre datos no vistos, con lo que se maximiza la capacidad de generalización.

Ejercicio 6. Poda mediante el error pesimista

Considere el siguiente subárbol: un nodo interno contiene 40 instancias del conjunto de entrenamiento, de las cuales 28 pertenecen a la clase “+” y 12 a la clase “-”. Dicho nodo se divide en cuatro nodos hoja, cuyas distribuciones de clase se detallan en la Tabla 4.

a) Calcule el error de entrenamiento del nodo antes de la división (tratándolo como hoja) y después de la división.

Antes de la división, el nodo se clasifica según la clase mayoritaria (“+”), lo que genera 12 errores sobre 40 instancias:

$$\text{Error de entrenamiento (antes)} = \frac{12}{40} = 0,30$$

Después de la división, cada hoja se clasifica según su clase mayoritaria (L1, L2 y L3 predicen “+”; L4 predice “−”), con un total de 11 errores resultantes:

$$\text{Error de entrenamiento (después)} = \frac{11}{40} = 0,275$$

La división produce una reducción del error de entrenamiento de 0,025.

b) *Calcule el error de generalización pesimista antes y después de la división, con una penalización de 0,5 por cada nodo hoja. El error pesimista se define como $(\text{errores} + k \cdot 0,5) / N$, donde k es el número de hojas y N el número de instancias.*

El error de generalización pesimista se define como $\frac{\text{errores} + k \times 0,5}{N}$, donde k es el número de hojas y N el número de instancias.

Antes de la división ($k = 1$):

$$\text{Error pesimista} = \frac{12 + 1 \times 0,5}{40} = \frac{12,5}{40} = 0,3125$$

Después de la división ($k = 4$):

$$\text{Error pesimista} = \frac{11 + 4 \times 0,5}{40} = \frac{13}{40} = 0,325$$

Aunque la división reduce el error de entrenamiento, el incremento en la complejidad del árbol provoca un aumento del error pesimista.

c) *A partir de la comparación de los errores pesimistas, ¿debería podarse el subárbol? Justifique su respuesta.*

Sí, el subárbol debería podarse, ya que el error pesimista aumenta de 0,3125 a 0,325 después de la división. Esto indica que la mayor complejidad del subárbol no mejora la capacidad de generalización del modelo.

d) *¿Qué relación tiene este procedimiento con el principio de parsimonia, conocido como la navaja de Occam?*

Este procedimiento está relacionado con el **principio de parsimonia** o navaja de Occam, que establece que, entre varias soluciones con desempeño similar, debe preferirse la más simple. En el contexto de los árboles de decisión, la poda busca eliminar divisiones que aumentan la complejidad del modelo sin aportar una mejora significativa en su capacidad de generalización, en favor de árboles más simples, menos propensos al sobreajuste y más fáciles de interpretar.

Ejercicio 7. Estrategias de control del sobreajuste

Responda de forma fundamentada a las siguientes consignas:

a) *Distinga la pre-poda (detención temprana) de la post-poda. Indique una ventaja y una desventaja de cada estrategia.*

La **pre-poda** consiste en detener el crecimiento del árbol antes de que se complete, mediante criterios que limitan su complejidad. Su principal ventaja es que reduce el costo computacional y evita la generación de árboles excesivamente grandes. Como desventaja, puede detener el crecimiento prematuramente y perder información relevante.

La **post-poda** consiste en generar primero el árbol completo y luego eliminar las ramas que no contribuyen significativamente a la capacidad de generalización. Su ventaja es que suele producir modelos más precisos, ya que evalúa el árbol completo antes de simplificarlo. Como desventaja, requiere un mayor costo computacional.

b) *Mencione tres criterios de pre-poda que se utilicen habitualmente en la inducción de árboles de decisión.*

Tres criterios de pre-poda de uso habitual son:

- Establecer una profundidad máxima del árbol.
- Exigir un número mínimo de instancias para dividir un nodo.
- Requerir una ganancia mínima de información (o reducción mínima de impureza) para realizar una división.

c) *Un colega afirma: “Mi árbol clasifica correctamente el 100 % de los datos de entrenamiento, por lo tanto, es un excelente modelo”. Critique esta afirmación de manera rigurosa.*

La afirmación no es necesariamente correcta. Un árbol que clasifica correctamente el 100 % de los datos de entrenamiento puede haber aprendido no solo los patrones relevantes, sino también el ruido presente en dichos datos; este fenómeno se conoce como sobreajuste (*overfitting*). Un error de entrenamiento nulo no garantiza una buena capacidad de generalización: para evaluar la calidad del modelo es necesario analizar su desempeño sobre datos no utilizados durante el entrenamiento.

d) *Explique cómo la validación cruzada de k pliegues permite estimar el error de generalización y por qué resulta preferible a una única partición de retención (holdout) cuando se dispone de pocos datos.*

La validación cruzada de k pliegues divide el conjunto de datos en k subconjuntos. En cada iteración, uno de ellos se utiliza para validación y los restantes para entrenamiento; el proceso se repite k veces y el error final corresponde al promedio de los resultados obtenidos. Este método permite obtener una estimación más robusta del error de generalización, ya que todas las observaciones participan tanto en entrenamiento como en validación. Cuando se dispone de pocos datos, resulta preferible a una única partición *holdout*, porque aprovecha mejor la información disponible y reduce la dependencia de una única división de los datos.

Parte D. Clasificadores Basados en Reglas

Ejercicio 8. Derivación de reglas a partir de un árbol de decisión

Considere el árbol de decisión obtenido en el Ejercicio 2, inducido sobre el Conjunto de Datos 1.

a) Derive el conjunto completo de reglas de clasificación, recorriendo cada camino desde la raíz hasta una hoja.

A partir del árbol de decisión, se deriva una regla por cada camino que va desde la raíz hasta una hoja.

Regla 1:

Si Ingreso = Alto, entonces Otorga = Sí.

Regla 2:

Si Ingreso = Medio e Historial = Bueno, entonces Otorga = Sí.

Regla 3:

Si Ingreso = Medio e Historial = Malo, entonces Otorga = No.

Regla 4:

Si Ingreso = Bajo e Historial = Malo, entonces Otorga = No.

Regla 5:

Si Ingreso = Bajo, Historial = Bueno y Deuda = Baja, entonces Otorga = Sí.

Regla 6:

Si Ingreso = Bajo, Historial = Bueno y Deuda = Alta, entonces Otorga = No.

b) Para cada regla, indique su cobertura (la cantidad de instancias del Conjunto de Datos 1 que satisface su antecedente) y su exactitud sobre dichas instancias.

La **cobertura** de una regla es la cantidad de instancias del conjunto de datos que cumplen el antecedente de esa regla.

La **exactitud** se calcula como:

$$\text{Exactitud} = \frac{\text{casos correctamente clasificados por la regla}}{\text{casos cubiertos por la regla}}$$

Tabla 6*Cobertura y exactitud de las reglas derivadas del árbol de decisión*

Regla	Antecedente	Cobertura	Exactitud
R1	Ingreso = Alto	4	100 %
R2	Ingreso = Medio e Historial = Bueno	3	100 %
R3	Ingreso = Medio e Historial = Malo	2	100 %
R4	Ingreso = Bajo e Historial = Malo	3	100 %
R5	Ingreso = Bajo e Historial = Bueno y Deuda = Baja	1	100 %
R6	Ingreso = Bajo e Historial = Bueno y Deuda = Alta	1	100 %

Nota. Elaboración propia a partir del árbol de decisión del Ejercicio 2.

c) *¿El conjunto de reglas resultante es mutuamente excluyente y exhaustivo? Justifique su respuesta.*

Sí. El conjunto de reglas es mutuamente excluyente porque cada instancia satisface una sola regla, dado que cada camino del árbol conduce a una única hoja. Es decir, una misma instancia no puede caer en dos hojas distintas al mismo tiempo.

También es exhaustivo porque las ramas del árbol cubren todos los valores posibles de los atributos considerados. Por lo tanto, toda instancia puede ser clasificada por alguna regla. En este caso, cualquier solicitante con **Ingreso = Alto**, **Ingreso = Medio** o **Ingreso = Bajo** termina en alguna hoja y recibe una clasificación.

d) *Clasifique los siguientes solicitantes nuevos: S1 (Ingreso = Medio, Historial = Malo, Deuda = Baja) y S2 (Ingreso = Bajo, Historial = Bueno, Deuda = Alta).*

Solicitante S1:

Ingreso = Medio, Historial = Malo, Deuda = Baja.

Como Ingreso = Medio e Historial = Malo, se aplica la **Regla 3**. Por lo tanto:

Otorga = No

Solicitante S2:

Ingreso = Bajo, Historial = Bueno, Deuda = Alta.

Como Ingreso = Bajo, Historial = Bueno y Deuda = Alta, se aplica la **Regla 6**. Por lo tanto:

Otorga = No

Ejercicio 9. Conjunto ordenado de reglas frente a votación

Un sistema antifraude clasifica las transacciones como “Fraude” o “Legítima” mediante el siguiente conjunto de reglas: $R1: (\text{Monto} = \text{Alto}) \vee (\text{PaisDistinto} = \text{Sí}) \rightarrow \text{Fraude}$; $R2: (\text{HoraInusual} = \text{Sí}) \rightarrow \text{Fraude}$; $R3: (\text{Monto} = \text{Bajo}) \rightarrow \text{Legítima}$; $R4: (\text{ClienteAntiguo} = \text{Sí}) \rightarrow \text{Legítima}$; Regla por defecto $\rightarrow \text{Legítima}$. Clasifique las tres transacciones de la Tabla 5 bajo dos esquemas: (i) un conjunto ordenado de reglas, es decir, una lista de decisión con prioridad $R1 > R2 > R3 > R4$; y (ii) una votación no ponderada de todas las reglas cuyo antecedente se satisface.

a) Indique la clase asignada a cada transacción en cada uno de los dos esquemas.

La clasificación de cada transacción bajo los dos esquemas se presenta en la siguiente tabla:

Tabla 7

Clasificación de transacciones bajo esquema ordenado y votación no ponderada

Transacción	Reglas satisfechas	Conjunto ordenado	Votación no ponderada
T1	$R1 \rightarrow \text{Fraude}$; $R2 \rightarrow \text{Fraude}$	Fraude ($R1$, primera coincidencia)	Fraude (2 votos Fraude, 0 Legítima)
T2	$R2 \rightarrow \text{Fraude}$; $R3 \rightarrow \text{Legítima}$; $R4 \rightarrow \text{Legítima}$	Fraude ($R2$ precede a $R3$ y $R4$)	Legítima (1 voto Fraude, 2 votos Legítima)
T3	Ninguna regla específica	Legítima (regla por defecto)	Legítima (regla por defecto)

Nota. Elaboración propia a partir de los datos del enunciado.

b) Señale en qué transacciones difieren los resultados de ambos esquemas y explique la causa de la discrepancia.

Los dos esquemas difieren únicamente en la transacción T2. En el conjunto ordenado, $R2$ ($\text{HoraInusual} = \text{Sí} \rightarrow \text{Fraude}$) precede a $R3$ ($\text{Monto} = \text{Bajo} \rightarrow \text{Legítima}$) y a $R4$ ($\text{ClienteAntiguo} = \text{Sí} \rightarrow \text{Legítima}$) en el orden de prioridades; al satisfacerse el antecedente de $R2$, la clasificación se detiene y T2 recibe la clase Fraude sin evaluar las reglas restantes. En el esquema de votación, las tres reglas cuyo antecedente se satisface emiten su voto de forma independiente: $R2$ aporta un voto a Fraude, mientras que $R3$ y $R4$ aportan un voto cada una a Legítima; la clase mayoritaria (2 contra 1) determina el resultado final como Legítima.

c) Discuta una ventaja y una desventaja de cada esquema de resolución de conflictos entre reglas.

El esquema de **conjunto ordenado** tiene como ventaja su eficiencia: la clasificación se detiene al encontrar la primera regla cuyo antecedente se satisface, sin necesidad de evaluar el resto. Su desventaja principal es que el orden de las reglas es determinante: una regla general ubicada antes que una más específica puede anularla, aun cuando la más específica sea más precisa para ese caso particular.

El esquema de **votación no ponderada** tiene como ventaja su robustez: al considerar todas las reglas aplicables de forma simultánea, el impacto de una regla individualmente incorrecta

queda atenuado por las restantes. Su desventaja es el mayor costo computacional, dado que cada transacción debe ser evaluada contra la totalidad de las reglas antes de emitir una decisión.

Ejercicio 10. Generación directa de reglas: cobertura secuencial

El Conjunto de Datos 3 contiene 10 puntos en un espacio bidimensional, descritos por los atributos continuos x_1 y x_2 , y etiquetados con la clase “+” o “-”.

a) Describa, con sus propias palabras, el algoritmo de cobertura secuencial para la generación directa de reglas.

El algoritmo de cobertura secuencial construye un conjunto de reglas de forma iterativa. En cada iteración, se busca la regla que mejor cubra instancias de la clase objetivo sin incluir instancias de otras clases, con el objetivo de maximizar una medida de calidad, como la precisión o la estimación de Laplace. Una vez identificada la regla, las instancias cubiertas se eliminan del conjunto de entrenamiento y el proceso se repite. El ciclo continúa hasta que todas las instancias de la clase objetivo han sido cubiertas o se alcanza un criterio de detención. Al finalizar, una regla por defecto asigna la clase mayoritaria a las instancias no cubiertas por ninguna regla específica (Tan et al., 2019).

b) Aplique el algoritmo para construir un conjunto de reglas que cubra la clase “+”. Cada regla debe expresarse como una conjunción de condiciones sobre x_1 y x_2 y no debe cubrir ningún punto de la clase “-”.

Los puntos del Conjunto de Datos 3 se distribuyen de la siguiente manera:

- Clase “+”: P1(1,1), P2(1,3), P3(2,2), P4(2,4), P5(6,1)
- Clase “-”: P6(4,4), P7(5,5), P8(5,3), P9(3,5), P10(6,5)

Iteración 1. La condición $x_1 \leq 2$ cubre P1, P2, P3 y P4, todos de clase “+”, sin incluir ningún punto de clase “-” (todos los negativos tienen $x_1 \geq 3$). Se incorpora la primera regla:

$$R1 : \text{SI } x_1 \leq 2 \Rightarrow \text{clase “+”}$$

Se eliminan P1, P2, P3 y P4. Positivos restantes: {P5(6,1)}.

Iteración 2. La condición $x_2 \leq 1$ cubre P5(6,1) sin incluir ningún negativo (todos tienen $x_2 \geq 3$). Se incorpora la segunda regla:

$$R2 : \text{SI } x_2 \leq 1 \Rightarrow \text{clase “+”}$$

No quedan positivos sin cubrir. El algoritmo concluye. El conjunto de reglas resultante es:

- $R1$: SI $x_1 \leq 2 \rightarrow$ clase “+”
- $R2$: SI $x_2 \leq 1 \rightarrow$ clase “+”
- Regla por defecto: clase “-”

c) Indique cuántas reglas fueron necesarias y discuta qué consecuencias tendría permitir reglas con una cobertura muy baja.

La construcción requirió **dos reglas**. Si se permitieran reglas con cobertura muy baja (por ejemplo, una sola instancia por regla), el algoritmo podría generar tantas reglas como instancias positivas existan, con lo que memorizaría el conjunto de entrenamiento en lugar de capturar los patrones subyacentes. Este fenómeno se conoce como sobreajuste (*overfitting*): el clasificador resultante tendría buen desempeño sobre los datos de entrenamiento pero escasa capacidad de generalización frente a instancias nuevas. Además, un número elevado de reglas excesivamente específicas reduce la interpretabilidad del modelo y dificulta su mantenimiento.

Parte E. Preguntas Teóricas de Desarrollo

Responda de forma clara, precisa y fundamentada. Se valorará el uso correcto de la terminología técnica y la articulación de los conceptos.

1. *Explique la diferencia entre aprendizaje supervisado y no supervisado, y ubique la tarea de clasificación dentro de esta taxonomía.*

La diferencia fundamental entre el aprendizaje supervisado y el no supervisado radica en la disponibilidad de etiquetas de clase durante el entrenamiento. En el aprendizaje supervisado, cada instancia del conjunto de entrenamiento está asociada a una etiqueta conocida, y el modelo aprende a mapear instancias a salidas mediante la minimización de un error sobre dichos ejemplos etiquetados (Tan et al., 2019).

En el aprendizaje no supervisado no se dispone de etiquetas; el objetivo consiste en descubrir estructura latente en los datos, tal como agrupamientos naturales (*clustering*) o patrones de co-ocurrencia (reglas de asociación).

La clasificación es una tarea de aprendizaje supervisado: a partir de un conjunto de instancias etiquetadas, se induce un modelo capaz de predecir la clase de nuevas instancias no vistas durante el entrenamiento. La variable de salida es discreta y categórica, lo que distingue a la clasificación de la regresión, cuya salida es continua.

2. *¿Por qué el error de entrenamiento (error de resustitución) no constituye un estimador fiable del desempeño de un modelo sobre datos nuevos?*

El error de resustitución evalúa el modelo sobre el mismo conjunto de datos empleado para su entrenamiento, lo que permite al modelo memorizar instancias particulares, incluido el ruido presente en los datos, en lugar de aprender los patrones generalizables subyacentes. Este fenómeno se denomina sobreajuste. Un árbol completamente expandido sin poda puede alcanzar un error de resustitución nulo y al mismo tiempo exhibir un error de generalización elevado sobre datos no vistos (Tan et al., 2019).

El error de resustitución constituye, en consecuencia, un estimador optimista y sesgado del desempeño real del modelo. Para obtener una estimación confiable del error de generalización es necesario evaluarlo sobre datos independientes del entrenamiento, por ejemplo mediante un conjunto de prueba o mediante validación cruzada.

3. *Compare el índice de Gini, la entropía y el error de clasificación como medidas de impureza. ¿Por qué la ganancia de información tiende a favorecer los atributos con muchos valores y de qué modo lo corrige la razón de ganancia?*

Las tres medidas de impureza más empleadas en la inducción de árboles de decisión difieren en su sensibilidad a las distribuciones de clase:

- **Error de clasificación:** $1 - \max_c p(c|t)$. Simple pero poco sensible a cambios en las probabilidades de clase, dado que solo considera la clase más frecuente.
- **Entropía:** $-\sum_c p(c|t) \log_2 p(c|t)$. Penaliza con mayor intensidad las distribuciones uniformes y resulta más sensible a cambios en las clases minoritarias.

- **Índice de Gini:** $1 - \sum_c p(c|t)^2$. De comportamiento similar a la entropía, es computacionalmente eficiente.

La ganancia de información tiende a favorecer atributos con muchos valores distintos porque la partición en numerosos subconjuntos pequeños puede producir nodos aparentemente puros, aunque demasiado específicos para generalizar. La razón de ganancia (*gain ratio*) corrige este sesgo al dividir la ganancia por la información de la partición (*split information*), con lo que penaliza los atributos que generan un número elevado de ramas (Tan et al., 2019).

4. *Enuncie y justifique las principales ventajas y limitaciones de los árboles de decisión como clasificadores.*

Los árboles de decisión presentan las siguientes ventajas principales:

- **Interpretabilidad:** la estructura del árbol se traduce directamente en reglas de tipo «si... entonces» legibles sin necesidad de formación técnica especializada.
- **Ausencia de requisitos de preprocesamiento:** no requieren normalización ni estandarización de atributos y admiten de forma nativa tanto variables categóricas como continuas.
- **Robustez frente a atributos irrelevantes:** los atributos que no aportan ganancia de información no son seleccionados para ninguna partición y no afectan el modelo.

Entre sus principales limitaciones se destacan:

- **Propensión al sobreajuste:** sin poda, los árboles completamente expandidos memorizan los datos de entrenamiento y generalizan de forma deficiente.
- **Inestabilidad:** pequeñas variaciones en el conjunto de entrenamiento pueden producir árboles estructuralmente muy distintos, dado que la selección del atributo raíz condiciona toda la estructura posterior.
- **Fronteras de decisión paralelas a los ejes:** cada partición considera un único atributo, por lo que las fronteras diagonales o curvas solo pueden aproximarse mediante un número elevado de divisiones consecutivas.

5. *Explique el compromiso entre sesgo y varianza y relaciónelo con los fenómenos de subajuste y sobreajuste.*

El error esperado de generalización de un modelo se descompone en tres componentes: el ruido irreducible del problema, el sesgo al cuadrado y la varianza.

- El **sesgo** cuantifica el error sistemático introducido por las suposiciones simplificadoras del modelo. Un modelo con sesgo elevado clasifica erróneamente de forma consistente, independientemente del conjunto de entrenamiento utilizado.
- La **varianza** cuantifica la sensibilidad del modelo a las fluctuaciones en los datos de entrenamiento. Un modelo con varianza elevada ajusta los datos en detalle pero generaliza de forma deficiente (Tan et al., 2019).

El **subajuste** corresponde a sesgo elevado y varianza baja: el modelo es demasiado simple para capturar los patrones relevantes y comete errores altos tanto en entrenamiento como en validación. El **sobreajuste** corresponde a sesgo bajo y varianza elevada: el modelo se ajusta en exceso a los datos de entrenamiento, al haber incorporado el ruido, y su desempeño se

degrada sobre datos no vistos. La complejidad óptima minimiza la suma de sesgo² y varianza, con lo que se maximiza la capacidad de generalización.

6. *Compare los clasificadores basados en reglas con los árboles de decisión en términos de expresividad, interpretabilidad y facilidad de mantenimiento.*

Desde el punto de vista de la **expresividad**, un clasificador basado en reglas generado mediante cobertura secuencial puede ser más expresivo que un árbol de decisión, dado que cada regla no está condicionada por la jerarquía de particiones establecida en los nodos superiores. Cada regla puede capturar una región del espacio de instancias de forma independiente (Tan et al., 2019).

En cuanto a la **interpretabilidad**, ambas representaciones son intrínsecamente legibles. No obstante, las reglas individuales pueden ser más fáciles de evaluar en forma aislada por expertos de dominio, quienes pueden validar cada condición sin necesidad de recorrer una estructura jerárquica completa.

Respecto a la **facilidad de mantenimiento**, los clasificadores basados en reglas presentan una ventaja clara: es posible agregar, modificar o eliminar reglas individuales sin reconstruir el modelo completo. En un árbol de decisión, la modificación de una partición en un nodo interno de alta jerarquía afecta todos los subárboles dependientes.

7. *¿En qué consiste el problema del desbalance de clases? ¿Por qué la exactitud resulta una métrica inadecuada en ese contexto y qué alternativas propondría?*

El desbalance de clases se produce cuando la distribución de instancias entre clases es marcadamente asimétrica, como en la detección de fraude o el diagnóstico de enfermedades poco frecuentes. En ese contexto, la exactitud resulta una métrica inadecuada porque un clasificador trivial que predice siempre la clase mayoritaria obtiene valores elevados de exactitud sin poseer capacidad discriminativa alguna (Tan et al., 2019).

Entre las alternativas más apropiadas se encuentran:

- La **sensibilidad** y la **precisión**, focalizadas en la clase minoritaria.
- La **medida F1**, que armoniza precisión y sensibilidad en un único valor.
- El **estadístico Kappa de Cohen**, que ajusta el acuerdo observado por el efecto del azar.
- El **área bajo la curva ROC (AUC-ROC)**, que evalúa la capacidad discriminativa en todos los umbrales posibles.

Desde el punto de vista del preprocesamiento, el sobremuestreo de la clase minoritaria (por ejemplo, mediante SMOTE) o el submuestreo de la mayoritaria permiten rebalancear el conjunto de entrenamiento.

8. *Un clasificador basado en reglas y un árbol de decisión pueden representar el mismo conocimiento. Discuta en qué situaciones preferiría cada una de las dos representaciones.*

Aunque un clasificador basado en reglas y un árbol de decisión pueden representar el mismo conocimiento, cada representación resulta preferible en distintos contextos.

El **árbol de decisión** es preferible cuando el proceso de decisión es inherentemente jerárquico (las decisiones tempranas condicionan las posteriores) y cuando el objetivo es visualizar la lógica de clasificación completa en una única estructura compacta, o cuando se requiere auditar el proceso de forma integral.

El **clasificador basado en reglas** es preferible cuando distintos subconjuntos del espacio de instancias siguen patrones independientes que no pueden organizarse naturalmente en una jerarquía sin introducir particiones redundantes. También resulta más adecuado cuando la lógica debe comunicarse a expertos de dominio que evalúan criterios en forma individual, o cuando el modelo debe actualizarse de manera incremental mediante la incorporación, modificación o eliminación de reglas sin afectar el resto del clasificador.

Parte F. Práctica en R

Esta parte se resuelve en R. Se utilizará el conjunto de datos *PimaIndiansDiabetes* del paquete *mlbench* (768 observaciones; ocho atributos predictores numéricos y el atributo de clase *diabetes*, con valores *pos* y *neg*). El siguiente fragmento prepara el entorno de trabajo:

```
pkgs <- c("rpart", "rpart.plot", "caret", "C50", "mlbench", "RWeka",
  ↪ "ggplot2")
faltan <- pkgs[!(pkgs %in% installed.packages()[, "Package"])]
if (length(faltan)) install.packages(faltan, repos =
  ↪ "https://cloud.r-project.org")

# El algoritmo PART (Ejercicio 13 d) usa RWeka, que requiere un JDK
  ↪ accesible vía rJava.
# En macOS se resuelve JAVA_HOME de forma automática; en otros sistemas se
  ↪ respeta el definido.
if (Sys.getenv("JAVA_HOME") == "" && file.exists("/usr/libexec/java_home"))
  ↪ {
  jh <- tryCatch(system("/usr/libexec/java_home", intern = TRUE), error =
  ↪ function(e) "")
  if (length(jh) && nzchar(jh)) Sys.setenv(JAVA_HOME = jh)
}

library(rpart); library(rpart.plot); library(caret)
library(C50); library(mlbench); library(RWeka); library(ggplot2)
data(PimaIndiansDiabetes, package = "mlbench")
set.seed(2026)
df <- PimaIndiansDiabetes
```

El conjunto registra 768 observaciones de mujeres de la etnia Pima, descritas por ocho atributos predictores numéricos (número de embarazos, glucosa, presión arterial, pliegue cutáneo del tríceps, insulina, índice de masa corporal, función de pedigrí diabético y edad) y el atributo de clase *diabetes*, con niveles *neg* y *pos*. La clase está moderadamente desbalanceada: solo el 34.9 % de los casos corresponde a la clase positiva. Por ese motivo la evaluación no se apoya solo en la exactitud; también se reportan la sensibilidad, la especificidad y el estadístico Kappa, menos sensibles al desbalance.

Tabla 8

Distribución de la clase en PimaIndiansDiabetes

Clase	Frecuencia	Proporción
neg	500	65.1 %
pos	268	34.9 %

Nota. Elaboración propia a partir del paquete *mlbench*.

Ejercicio 11. Árboles de decisión con rpart

a) Cargue los datos y particiónelos en un 70 % de entrenamiento y un 30 % de prueba mediante `caret::createDataPartition`. Fije la semilla del generador de números aleatorios para garantizar la reproducibilidad.

```
set.seed(2026)
idx  <- createDataPartition(df$diabetes, p = 0.70, list = FALSE)
train <- df[idx, ]
test  <- df[-idx, ]
c(entrenamiento = nrow(train), prueba = nrow(test))
```

```
## entrenamiento      prueba
##           538         230
```

La función `caret::createDataPartition` realiza un muestreo estratificado por la variable de clase, de modo que conserva en cada subconjunto la proporción original de `pos` y `neg`. La estratificación es deseable en un problema desbalanceado, porque evita que el azar deje al conjunto de prueba con muy pocos positivos. La semilla se fija con `set.seed(2026)` inmediatamente antes de la partición.

b) Induzca un árbol de decisión sobre el conjunto de entrenamiento con la función `rpart()` y visualícelo con `rpart.plot()`.

```
set.seed(2026)
tree_def <- rpart(diabetes ~ ., data = train, method = "class")
rpart.plot(tree_def, type = 2, extra = 104, fallen.leaves = TRUE,
  ↪ box.palette = "BuGn")
```

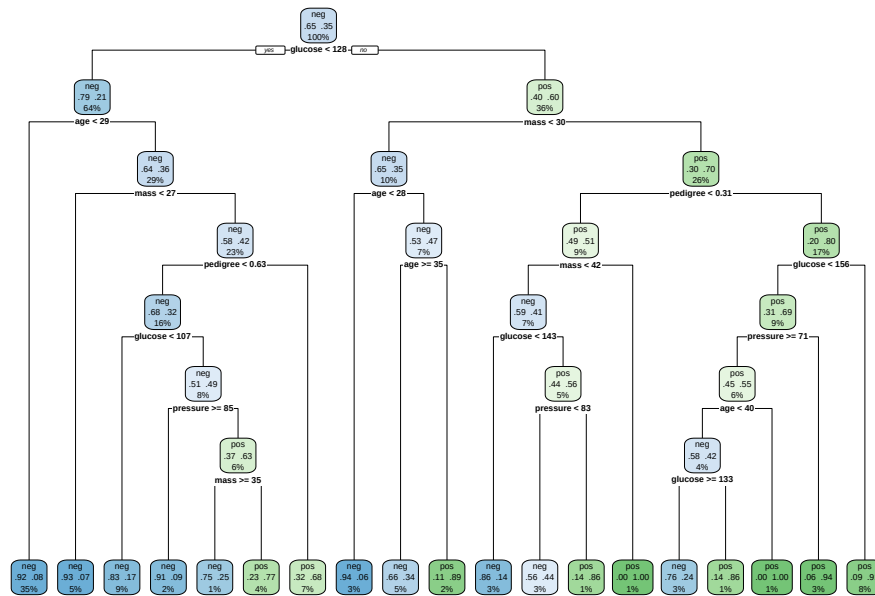


Figura 4: Árbol de decisión por defecto inducido con `rpart` ($cp = 0.01$).

El árbol se induce con los hiperparámetros por defecto de `rpart`, entre ellos el parámetro de complejidad $cp = 0.01$, que opera como criterio de pre poda: solo admite particiones que reduzcan el error relativo en al menos un uno por ciento. El árbol resultante tiene 19 nodos hoja. La glucosa es el atributo seleccionado en la raíz, hecho coherente con el dominio clínico, donde constituye el predictor más informativo de la diabetes.

c) Calcule el error de resustitución (sobre el conjunto de entrenamiento) y el error de generalización (sobre el conjunto de prueba) mediante `caret::confusionMatrix()`. Compare ambos valores y comente.

```

pred_train <- predict(tree_def, train, type = "class")
pred_test  <- predict(tree_def, test,  type = "class")
cm_train   <- confusionMatrix(pred_train, train$diabetes, positive = "pos")
cm_test    <- confusionMatrix(pred_test,  test$diabetes,  positive = "pos")
acc_train  <- cm_train$overall["Accuracy"]
acc_test   <- cm_test$overall["Accuracy"]

```

Tabla 9

Error de resustitución frente a error de generalización (árbol por defecto)

Conjunto	Exactitud	Tasa de error
Entrenamiento (resustitución)	0.8532	0.1468
Prueba (generalización)	0.7391	0.2609

Nota. Elaboración propia a partir del árbol inducido con `rpart`.

El error de resustitución es de 14.7 % y el error de generalización sube a 26.1 %. La diferencia entre ambos refleja el optimismo del error de entrenamiento: el modelo se ajusta en parte a las particularidades de la muestra de entrenamiento. La indicación `positive = "pos"` es deliberada, porque por defecto `confusionMatrix` toma como clase positiva el primer nivel del factor (`neg`); dado que el objetivo clínico es la detección de diabetes, la clase de interés debe ser `pos`.

d) Informe la exactitud, la sensibilidad, la especificidad y el estadístico Kappa obtenidos sobre el conjunto de prueba.

Tabla 10

Métricas de desempeño sobre el conjunto de prueba (árbol por defecto)

	Métrica	Valor
Accuracy	Exactitud	0.7391
Sensitivity	Sensibilidad	0.5500
Specificity	Especificidad	0.8400
Kappa	Kappa de Cohen	0.4041

Nota. La clase positiva es `pos` (paciente diabética).

Sobre el conjunto de prueba el árbol alcanza una exactitud de 73.9 % y un Kappa de 0.404, indicador de una concordancia moderada por encima del azar. La sensibilidad es de apenas 55.0 % frente a una especificidad de 84.0 %: el modelo detecta bien a las pacientes sanas, pero no identifica a buena parte de las diabéticas reales (falsos negativos). Ese desbalance entre sensibilidad y especificidad es el costo que el árbol por defecto asume para maximizar la exactitud global, y la sensibilidad es la métrica más relevante en un tamiz clínico.

Ejercicio 12. Sobreajuste y poda en R

a) Induzca un árbol «completo» fijando los hiperparámetros `rpart.control(minsplit = 2, cp = 0)` y compare su error de resustitución con su error de generalización. ¿Qué evidencia de sobreajuste observa?

```
n_hojas <- function(modelo) {
  sum(modelo$frame$var == "<leaf>")
}
set.seed(2026)
tree_full <- rpart(diabetes ~ ., data = train, method = "class",
  control = rpart.control(minsplit = 2, cp = 0))
pred_full_train <- predict(tree_full, train, type = "class")
pred_full_test  <- predict(tree_full, test,  type = "class")
acc_full_train  <- confusionMatrix(pred_full_train, train$diabetes,
  positive = "pos")$overall["Accuracy"]
acc_full_test   <- confusionMatrix(pred_full_test, test$diabetes,
```


b) Utilice `caret::train()` con validación cruzada de 10 pliegues para seleccionar el valor óptimo del parámetro de complejidad `cp`.

```
set.seed(2026)
ctrl <- trainControl(method = "cv", number = 10)
grid <- expand.grid(cp = seq(0, 0.05, by = 0.002))
fit_cv <- train(diabetes ~ ., data = train, method = "rpart",
               trControl = ctrl, tuneGrid = grid)
cp_opt <- fit_cv$bestTune$cp
cp_opt
```

```
## [1] 0.006
```

La función `caret::train` reentrena el árbol para cada valor de `cp` de la grilla y estima su desempeño por validación cruzada de 10 pliegues: divide el entrenamiento en diez partes, entrena con nueve y evalúa en la restante, con rotación. El error medio sobre los diez pliegues es un estimador del error de generalización más estable que el de una única partición. El valor seleccionado, de máxima exactitud media, es `cp = 0.006`.

c) Genere la curva del error estimado por validación cruzada en función de `cp` e interprétela.

```
res <- fit_cv$results
res$error_cv <- 1 - res$Accuracy
ggplot(res, aes(x = cp, y = error_cv)) +
  geom_line(color = "steelblue") +
  geom_point(color = "steelblue") +
  geom_point(data = res[res$cp == cp_opt, ], color = "red", size = 3) +
  geom_vline(xintercept = cp_opt, linetype = "dashed", color = "red") +
  labs(x = "Parámetro de complejidad (cp)",
       y = "Error de clasificación (validación cruzada)") +
  theme_minimal(base_size = 12)
```

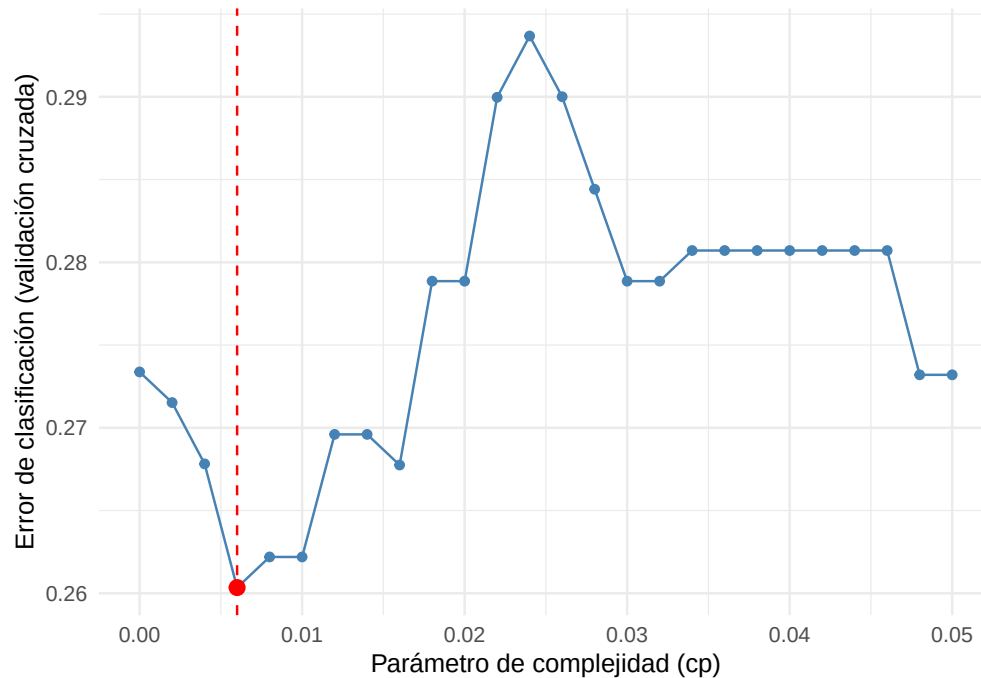


Figura 6: Error de clasificación estimado por validación cruzada (10 pliegues) en función de cp ; el punto rojo marca el cp óptimo.

La curva adopta la forma característica del compromiso entre sesgo y varianza. Con cp próximo a cero el árbol es demasiado complejo, sobreajusta y el error de validación cruzada es alto. A medida que cp aumenta, la poda elimina ramas espurias y el error desciende hasta un mínimo en $cp = 0.006$. Pasado ese punto, un cp demasiado grande poda en exceso (subajuste) y el error vuelve a subir. El mínimo de la curva identifica la complejidad que mejor generaliza.

d) Compare el desempeño del árbol completo, el árbol por defecto y el árbol podado sobre el conjunto de prueba. Extraiga una conclusión.

```
tree_pruned <- prune(tree_full, cp = ifelse(cp_opt == 0, 1e-6, cp_opt))
n_hojas_pruned <- n_hojas(tree_pruned)
arboles <- list(`Árbol completo (cp = 0)` = tree_full,
               `Árbol por defecto (cp = 0.01)` = tree_def,
               `Árbol podado (cp óptimo)` = tree_pruned)
comp <- data.frame(
  Modelo = names(arboles),
  Hojas = sapply(arboles, n_hojas),
  t(sapply(arboles, function(m) metrics_test(predict(m, test, type =
    ↪ "class")))),
  row.names = NULL, check.names = FALSE)
```

Tabla 12*Desempeño sobre el conjunto de prueba de los tres árboles*

Modelo	Hojas	Exactitud	Sensibilidad	Especificidad	Kappa
Árbol completo ($cp = 0$)	98	0.6783	0.5375	0.7533	0.2908
Árbol por defecto ($cp = 0.01$)	19	0.7391	0.5500	0.8400	0.4041
Árbol podado (cp óptimo)	35	0.7435	0.5375	0.8533	0.4087

Nota. Elaboración propia.

El árbol podado tiene 35 hojas, frente a las 98 del completo, y es el que mejor generaliza de los tres, con la mayor exactitud y el mayor Kappa sobre prueba. La conclusión es directa: más complejidad no implica mejor desempeño. El árbol completo sobreajusta, el árbol por defecto aplica ya una prepoda razonable y la poda guiada por validación cruzada encuentra el punto de complejidad que minimiza el error de generalización estimado. El procedimiento recomendado consiste en dejar crecer el árbol y luego podarlo con un cp elegido por validación cruzada.

Ejercicio 13. Clasificador basado en reglas en R

a) Induzca un clasificador basado en reglas con el algoritmo *C5.0* (paquete *C50*), con la opción `rules = TRUE`, sobre el mismo conjunto de entrenamiento del Ejercicio 11.

```
set.seed(2026)
c5_rules <- C5.0(x = train[, setdiff(names(train), "diabetes")],
                 y = train$diabetes, rules = TRUE)
c5_rules

##
## Call:
## C5.0.default(x = train[, setdiff(names(train), "diabetes")], y
## = train$diabetes, rules = TRUE)
##
## Rule-Based Model
## Number of samples: 538
## Number of predictors: 8
##
## Number of Rules: 22
##
## Non-standard options: attempt to group attributes
```

Con `rules = TRUE`, *C5.0* primero induce un árbol y luego lo convierte en un conjunto de reglas que después simplifica, con poda de condiciones redundantes y eliminación de reglas que no aportan. El modelo resultante contiene 22 reglas.

b) Inspeccione el conjunto de reglas generado con `summary()` e interprete dos de las reglas obtenidas.


```
summary(c5_rules)
```

```
##
## Call:
## C5.0.default(x = train[, setdiff(names(train), "diabetes")], y
##   = train$diabetes, rules = TRUE)
##
##
## C5.0 [Release 2.07 GPL Edition]      Tue Jun 23 22:43:03 2026
## -----
##
## Class specified by attribute `outcome'
##
## Read 538 cases (9 attributes) from undefined.data
##
## Rules:
##
## Rule 1: (89/1, lift 1.5)
##   glucose <= 127
##   mass > 0
##   mass <= 26.9
##   ->  class neg  [0.978]
##
## Rule 2: (19, lift 1.5)
##   glucose <= 127
##   insulin > 32
##   insulin <= 85
##   age > 28
##   ->  class neg  [0.952]
##
## Rule 3: (68/4, lift 1.4)
##   glucose <= 127
##   pedigree <= 0.204
##   ->  class neg  [0.929]
##
## Rule 4: (120/8, lift 1.4)
##   glucose <= 99
##   pedigree <= 0.711
##   ->  class neg  [0.926]
##
## Rule 5: (189/16, lift 1.4)
##   glucose <= 127
##   age <= 28
##   ->  class neg  [0.911]
```

```
##
## Rule 6: (20/1, lift 1.4)
## glucose > 131
## glucose <= 157
## pressure > 70
## mass <= 46.8
## age > 26
## age <= 41
## -> class neg [0.909]
##
## Rule 7: (12/1, lift 1.3)
## mass <= 29.9
## age > 60
## -> class neg [0.857]
##
## Rule 8: (418/119, lift 1.1)
## pregnant <= 6
## -> class neg [0.714]
##
## Rule 9: (25, lift 2.8)
## glucose > 157
## pressure > 70
## mass > 29.9
## age <= 49
## -> class pos [0.963]
##
## Rule 10: (16, lift 2.7)
## glucose > 157
## pressure > 70
## triceps <= 30
## mass > 29.9
## -> class pos [0.944]
##
## Rule 11: (15, lift 2.7)
## glucose > 127
## glucose <= 157
## mass > 29.9
## pedigree > 0.222
## age > 41
## -> class pos [0.941]
##
## Rule 12: (12, lift 2.7)
## glucose > 99
## glucose <= 127
## pressure <= 86
```

```
## triceps <= 24
## mass > 26.9
## mass <= 30.8
## pedigree > 0.204
## pedigree <= 0.761
## age > 28
## -> class pos [0.929]
##
## Rule 13: (10, lift 2.6)
## pregnant <= 6
## glucose <= 127
## pressure <= 86
## triceps <= 24
## mass > 26.9
## pedigree > 0.204
## pedigree <= 0.761
## age > 34
## -> class pos [0.917]
##
## Rule 14: (22/1, lift 2.6)
## glucose > 127
## pressure <= 61
## mass > 29.9
## -> class pos [0.917]
##
## Rule 15: (10, lift 2.6)
## glucose > 127
## glucose <= 131
## mass > 29.9
## pedigree > 0.331
## -> class pos [0.917]
##
## Rule 16: (9, lift 2.6)
## mass > 46.8
## -> class pos [0.909]
##
## Rule 17: (28/2, lift 2.6)
## glucose > 127
## pressure <= 70
## mass > 29.9
## pedigree > 0.331
## age <= 41
## -> class pos [0.900]
##
## Rule 18: (7, lift 2.5)
```

```
## glucose > 143
## mass <= 29.9
## age > 41
## age <= 60
## -> class pos [0.889]
##
## Rule 19: (15/1, lift 2.5)
## glucose > 157
## pressure <= 70
## triceps > 27
## -> class pos [0.882]
##
## Rule 20: (12/1, lift 2.5)
## glucose > 99
## glucose <= 118
## pressure <= 86
## triceps > 24
## mass > 26.9
## pedigree > 0.496
## age > 28
## -> class pos [0.857]
##
## Rule 21: (9/1, lift 2.3)
## glucose > 127
## mass <= 29.9
## age > 27
## age <= 35
## -> class pos [0.818]
##
## Rule 22: (278/143, lift 1.4)
## age > 28
## -> class pos [0.486]
##
## Default class: neg
##
##
## Evaluation on training data (538 cases):
##
##           Rules
##  -----
##      No      Errors
##
##      22    52( 9.7%)  <<
##
##
```

```
##      (a)   (b)   <-classified as
##      ----   ----
##      333    17   (a): class neg
##      35    153  (b): class pos
##
##
## Attribute usage:
##
## 92.94% age
## 78.25% glucose
## 77.70% pregnant
## 47.03% mass
## 45.17% pedigree
## 22.30% pressure
## 10.97% triceps
## 3.53% insulin
##
##
## Time: 0.0 secs
```

Cada regla se reporta como (n/m, lift L), donde **n** es la cobertura (instancias de entrenamiento que satisfacen el antecedente), **m** los errores dentro de esa cobertura y **lift** la mejora respecto de la probabilidad base de la clase. Se interpretan dos reglas representativas:

- Regla de alta precisión y baja cobertura (Regla 1): **glucose** ≤ 127 junto con **mass** ≤ 26.9 conducen a la clase **neg**. Cubre 89 pacientes de entrenamiento con un solo error (confianza cercana a 0.98, lift 1.5). Tiene sentido clínico, ya que una glucosa no elevada junto con un bajo índice de masa corporal son indicadores fuertes de ausencia de diabetes. Es una regla muy confiable, aunque específica.
- Regla de alta cobertura y baja precisión (Regla 8): **pregnant** ≤ 6 conduce a la clase **neg**. Cubre 418 casos con 119 errores (confianza cercana a 0.71, lift 1.1). Es casi una regla por defecto, ya que apenas mejora sobre la predicción de la clase mayoritaria. El contraste entre ambas reglas ilustra la diferencia entre reglas específicas y confiables frente a reglas generales y débiles.

El bloque **Attribute usage** de la salida indica el porcentaje de casos cuya clasificación recurre a cada atributo: encabezan la lista **age** (cerca del 93 %), **glucose** (cerca del 78 %) y **pregnant** (cerca del 78 %), seguidos de **mass** y **pedigree**, lo que confirma su relevancia.

c) *Evalúe el clasificador de reglas sobre el conjunto de prueba y compare su desempeño con el del árbol de decisión del Ejercicio 11.*

```
set.seed(2026)

c5_rules <- C5.0(
  diabetes ~ .,
  data = train,
```

```
rules = TRUE
)

summary(c5_rules)

##
## Call:
## C5.0.formula(formula = diabetes ~ ., data = train, rules = TRUE)
##
##
## C5.0 [Release 2.07 GPL Edition]      Tue Jun 23 22:43:03 2026
## -----
##
## Class specified by attribute `outcome'
##
## Read 538 cases (9 attributes) from undefined.data
##
## Rules:
##
## Rule 1: (89/1, lift 1.5)
##  glucose <= 127
##  mass > 0
##  mass <= 26.9
##  ->  class neg  [0.978]
##
## Rule 2: (19, lift 1.5)
##  glucose <= 127
##  insulin > 32
##  insulin <= 85
##  age > 28
##  ->  class neg  [0.952]
##
## Rule 3: (68/4, lift 1.4)
##  glucose <= 127
##  pedigree <= 0.204
##  ->  class neg  [0.929]
##
## Rule 4: (120/8, lift 1.4)
##  glucose <= 99
##  pedigree <= 0.711
##  ->  class neg  [0.926]
##
## Rule 5: (189/16, lift 1.4)
##  glucose <= 127
```

```
## age <= 28
## -> class neg [0.911]
##
## Rule 6: (20/1, lift 1.4)
## glucose > 131
## glucose <= 157
## pressure > 70
## mass <= 46.8
## age > 26
## age <= 41
## -> class neg [0.909]
##
## Rule 7: (12/1, lift 1.3)
## mass <= 29.9
## age > 60
## -> class neg [0.857]
##
## Rule 8: (418/119, lift 1.1)
## pregnant <= 6
## -> class neg [0.714]
##
## Rule 9: (25, lift 2.8)
## glucose > 157
## pressure > 70
## mass > 29.9
## age <= 49
## -> class pos [0.963]
##
## Rule 10: (16, lift 2.7)
## glucose > 157
## pressure > 70
## triceps <= 30
## mass > 29.9
## -> class pos [0.944]
##
## Rule 11: (15, lift 2.7)
## glucose > 127
## glucose <= 157
## mass > 29.9
## pedigree > 0.222
## age > 41
## -> class pos [0.941]
##
## Rule 12: (12, lift 2.7)
## glucose > 99
```

```
## glucose <= 127
## pressure <= 86
## triceps <= 24
## mass > 26.9
## mass <= 30.8
## pedigree > 0.204
## pedigree <= 0.761
## age > 28
## -> class pos [0.929]
##
## Rule 13: (10, lift 2.6)
## pregnant <= 6
## glucose <= 127
## pressure <= 86
## triceps <= 24
## mass > 26.9
## pedigree > 0.204
## pedigree <= 0.761
## age > 34
## -> class pos [0.917]
##
## Rule 14: (22/1, lift 2.6)
## glucose > 127
## pressure <= 61
## mass > 29.9
## -> class pos [0.917]
##
## Rule 15: (10, lift 2.6)
## glucose > 127
## glucose <= 131
## mass > 29.9
## pedigree > 0.331
## -> class pos [0.917]
##
## Rule 16: (9, lift 2.6)
## mass > 46.8
## -> class pos [0.909]
##
## Rule 17: (28/2, lift 2.6)
## glucose > 127
## pressure <= 70
## mass > 29.9
## pedigree > 0.331
## age <= 41
## -> class pos [0.900]
```



```
##
## Rule 18: (7, lift 2.5)
##  glucose > 143
##  mass <= 29.9
##  age > 41
##  age <= 60
##  ->  class pos  [0.889]
##
## Rule 19: (15/1, lift 2.5)
##  glucose > 157
##  pressure <= 70
##  triceps > 27
##  ->  class pos  [0.882]
##
## Rule 20: (12/1, lift 2.5)
##  glucose > 99
##  glucose <= 118
##  pressure <= 86
##  triceps > 24
##  mass > 26.9
##  pedigree > 0.496
##  age > 28
##  ->  class pos  [0.857]
##
## Rule 21: (9/1, lift 2.3)
##  glucose > 127
##  mass <= 29.9
##  age > 27
##  age <= 35
##  ->  class pos  [0.818]
##
## Rule 22: (278/143, lift 1.4)
##  age > 28
##  ->  class pos  [0.486]
##
## Default class: neg
##
##
## Evaluation on training data (538 cases):
##
##           Rules
##  -----
##      No      Errors
##
##      22    52( 9.7%)  <<
```

```
##
##
##      (a)   (b)   <-classified as
##      ----   ----
##      333    17   (a): class neg
##      35    153  (b): class pos
##
##
## Attribute usage:
##
## 92.94% age
## 78.25% glucose
## 77.70% pregnant
## 47.03% mass
## 45.17% pedigree
## 22.30% pressure
## 10.97% triceps
## 3.53% insulin
##
##
## Time: 0.0 secs

metrics_test <- function(predicciones) {
  cm <- confusionMatrix(predicciones, test$diabetes, positive = "pos")

  c(
    Accuracy = cm$overall["Accuracy"],
    Sensitivity = cm$byClass["Sensitivity"],
    Specificity = cm$byClass["Specificity"],
    Kappa = cm$overall["Kappa"]
  )
}

pred_c5 <- predict(c5_rules, test)
cm_c5 <- confusionMatrix(pred_c5, test$diabetes, positive = "pos")
comp_c5 <- rbind(
  `Árbol rpart (Ej. 11)` = metrics_test(pred_test),
  `Reglas C5.0 (Ej. 13)` = metrics_test(pred_c5))

round(comp_c5, 4)

##
## Accuracy.Accuracy Sensitivity.Sensitivity
## Árbol rpart (Ej. 11) 0.7391 0.55
## Reglas C5.0 (Ej. 13) 0.7826 0.65
##
## Specificity.Specificity Kappa.Kappa
## Árbol rpart (Ej. 11) 0.8400 0.4041
```

Reglas C5.0 (Ej. 13)

0.8533

0.5123

Tabla 13*Reglas C5.0 frente al árbol rpart sobre el conjunto de prueba*

	Exactitud	Sensibilidad	Especificidad	Kappa
Árbol rpart (Ej. 11)	0.7391	0.55	0.8400	0.4041
Reglas C5.0 (Ej. 13)	0.7826	0.65	0.8533	0.5123

Nota. Elaboración propia. La clase positiva es **pos**.

El clasificador de reglas C5.0 supera al árbol **rpart** del Ejercicio 11 en todas las métricas sobre prueba: exactitud 78.3 % frente a 73.9 %, Kappa 0.512 frente a 0.404 y sensibilidad 65.0 % frente a 55.0 %. La mejora en sensibilidad es la más valiosa en este problema, porque implica la detección de más diabéticas reales. C5.0 obtiene ese resultado por su poda basada en confianza y por la simplificación del conjunto de reglas, que produce un modelo a la vez compacto y más preciso.

d) *Discuta las diferencias de interpretabilidad entre el árbol de decisión y el conjunto de reglas. Como alternativa, puede emplearse el método “PART” mediante `caret::train()` con el paquete **RWeka**.*

```
set.seed(2026)
part_fit <- PART(diabetes ~ ., data = train)
pred_part <- predict(part_fit, test)
cm_part <- confusionMatrix(pred_part, test$diabetes, positive = "pos")
```

Tabla 14*Síntesis comparativa de todos los modelos sobre el conjunto de prueba*

	Exactitud	Sensibilidad	Especificidad	Kappa
Árbol completo (cp = 0)	0.6783	0.5375	0.7533	0.2908
Árbol por defecto (Ej. 11)	0.7391	0.5500	0.8400	0.4041
Árbol podado CV (Ej. 12)	0.7435	0.5375	0.8533	0.4087
Reglas C5.0 (Ej. 13)	0.7826	0.6500	0.8533	0.5123
Reglas PART (Ej. 13 d)	0.6957	0.4500	0.8267	0.2920

Nota. Elaboración propia. La clase positiva es **pos**.

El enunciado sugiere acceder a PART mediante `caret::train(method = "PART")`. Aquí se emplea la interfaz directa **RWeka::PART()**, equivalente y más transparente, dado que evita la capa de ajuste de hiperparámetros y de remuestreo que añade **caret**; de ese modo el modelo evaluado corresponde a PART con sus parámetros por defecto.

Un árbol de decisión expresa el conocimiento como una jerarquía única: la clasificación de una instancia recorre un solo camino de la raíz a una hoja, y las particiones son mutuamente excluyentes. Un conjunto de reglas (C5.0 o PART) expresa el conocimiento como una lista de reglas independientes del tipo «si... entonces clase». Cada regla es una afirmación autocontenida, fácil de validar con un experto del dominio, y el conjunto admite la incorporación o la eliminación de reglas sin reentrenar todo el modelo. La contrapartida es que un conjunto de reglas puede resultar redundante o ambiguo, porque varias reglas pueden cubrir una misma instancia y entrar en conflicto, lo que exige un mecanismo de resolución de conflictos (orden de prioridad o votación); el árbol evita esa ambigüedad por construcción, a costa de que las ramas profundas se vuelvan difíciles de leer.

En cuanto al desempeño sobre este conjunto, PART obtiene una exactitud de 69.6% y un Kappa de 0.292, por debajo de C5.0. PART genera un conjunto más compacto (9 reglas frente a las 22 de C5.0), pero esa menor cantidad de reglas no se traduce en mejor generalización: el tamaño del modelo, por sí solo, no determina su calidad. En síntesis, C5.0 con reglas es el mejor clasificador de los evaluados, dado que combina el desempeño más alto con una representación interpretable.

Referencias

- Tan, P.-N., Steinbach, M., Karpatne, A., & Kumar, V. (2019). *Introduction to data mining* (2.^a ed.). Pearson.
- Hahsler, M. (2025). *An R companion for Introduction to Data Mining*. Recuperado de https://mhahsler.github.io/Introduction_to_Data_Mining_R_Examples/
- Therneau, T., & Atkinson, B. (2025). *rpart: Recursive partitioning and regression trees* [Paquete de R]. <https://CRAN.R-project.org/package=rpart>
- Kuhn, M. (2024). *caret: Classification and regression training* [Paquete de R]. <https://CRAN.R-project.org/package=caret>